Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computación, Cartago
IC-6600 Principios de Sistemas Operativos
II-Semestre 2026, Prof. M.Sc. Esteban Arias Méndez

PROYECTO PROGRAMADO

Implementación de un Sistema de Memoria Compartida Distribuida (DSM) con Paginación bajo Demanda y Coherencia MSI

Modalidad: En parejas (2 estudiantes máximo)

Entorno de desarrollo: Linux (C / C++)

Herramientas obligatorias: Sockets TCP/IP, Pthreads, Semáforos POSIX, llamadas al sistema de memoria (`mmap`, `mprotect`) y manejo de señales (`sigaction`).

1. Introducción y Objetivos

En los sistemas distribuidos sin memoria física compartida, la comunicación tradicional se realiza mediante el paso de mensajes (sockets, RPC). Sin embargo, la **Memoria Compartida Distribuida (DSM)** emula un espacio de direcciones virtual compartido sobre un clúster de computadoras independientes conectadas por red.

El objetivo de este proyecto es construir un subsistema DSM en espacio de usuario sobre Linux. El sistema interceptará fallos de segmento a bajo nivel para mover páginas de memoria dinámicamente entre nodos a través de la red, manteniendo un protocolo de coherencia de caché.

2. Arquitectura General del Sistema

El clúster estará compuesto por N nodos (ejecutándose en distintas terminales de la misma máquina o en computadoras distintas en red). Cada nodo gestiona un bloque de memoria virtual reservado localmente que representa una parte de la "memoria compartida global"; el total de estos bloques de memoria local será el total de memoria compartida disponible del sistema.

La administración del sistema deberá ser distribuida. Cada vez que se agregue un nodo al sistema, este se comunicará con los nodos existentes para indicar la existencia de un nuevo nodo. Todos los nodos registrarán la información relevante de este nuevo nodo, como su dirección IP, la cantidad de memoria que aporta, etc. Todos los nodos tendrán una tabla con la información del resto de nodos. Los nodos deberán enviar de forma frecuente un mensaje al resto de los nodos para determinar si están en línea (disponibles o no, “heartbeat”). Para facilitar la comunicación, un archivo de configuración contendrá la información del puerto o puertos de comunicación para los nodos. Se utilizará un rango de puertos específico para cada pareja de trabajo. Este archivo de configuración podría ser modificado para restablecer el sistema con nuevos parámetros según se requiera.

Cada nodo administrará la tabla de páginas de su propia memoria. Cada nodo administrará también las peticiones de memoria de los procesos locales que soliciten su servicio. Es decir, si un programa quiere ejecutar alguna tarea en el sistema, lo pedirá al nodo local. Según la cantidad de memoria requerida, la tarea podría ser atendida sólo de forma local o bien, si la solicitud es de más memoria que la disponible localmente, el nodo se encargará de pedir más memoria a los otros nodos. Todos los nodos trabajarán de forma coordinada para determinar los espacios de memoria disponibles y los ocupados, asimismo, saber cuáles páginas pertenecen a cuáles procesos en cada nodo y el orden de dichas páginas que debe ser conservado para evitar inconsistencia de datos ante los procesos.

Cada nodo ejecutará la aplicación de validación requerida según los datos dados. El nodo deberá facilitar las páginas de memoria necesarias al proceso de forma transparente (sean páginas locales o distribuidas) para su ejecución.

Desde un nodo, un usuario podrá consultar al sistema información de datos locales o del sistema como tal. Debe ser posible consultar datos estadísticos del sistema, uso, consumo, disponibilidad. También se debe poder consultar la tabla de páginas local de cada nodo y de todo el sistema. Debe ser posible ver el uso y la asignación de las páginas en cada memoria de cada nodo y a qué procesos corresponden y en qué orden.

La información puede ser mostrada en una terminal regular del sistema en cada nodo en modo texto; en este caso, brinde un formato claro de salida, con mensajes claros sobre la información que se muestra. También puede ser posible generar estas consultas en formato HTML y visualizarlas en un navegador local en cada máquina. Acá debe tenerse en cuenta que, si hay varios nodos ejecutándose en la misma computadora, la web debe poder mostrar los datos de

cada nodo por separado. Desde cualquier nodo se podrá consultar información global del sistema.

Extras:

1. Se considerará con un valor extra del 10% a la nota la ejecución distribuida de procesos. Es decir, que desde un nodo se coordine la ejecución de procesos paralelos en otros nodos, usando la memoria distribuida en todo el sistema, pero procesando en múltiples nodos separados de forma local y paralela.

2. El considerar la seguridad del sistema ante posibles fallas o caída de algún nodo o la conexión de red, implementando un sistema de cierto nivel de tolerancia para mantener la integridad del sistema ante la pérdida de algún nodo de forma esporádica y no perder información ante su ausencia será considerado con un valor extra de 10%. Para esto deberán construir un sistema que brinde redundancia de datos para la recuperación ante pérdidas de algún nodo. Se deberá documentar el proceso, así como el nivel de tolerancia del sistema (cuántos nodos se pueden perder sin afectar con pérdida de datos), además indicar el costo de overhead para el sistema relacionado a la solución planteada.

2.1. Modelo de Consistencia (Protocolo MSI Simplificado)

Cada página de memoria (de tamaño estándar de 4 kb o el tamaño que el usuario configure para el sistema) en un nodo se encontrará al menos en uno de tres estados:

1. **Modified (M):** El nodo posee la versión más reciente y exclusiva de la página; otros nodos no la tienen o la tienen invalidada.
2. **Shared (S):** La página es de solo lectura; varios nodos pueden tener una copia sincronizada.
3. **Invalid (I):** La página no está disponible localmente o su contenido está desactualizado.

En caso de requerir más estados, podrá definirlos en el sistema. Los mismos deben ser justificados y documentados.

3. Especificaciones Técnicas y Módulos a Implementar

El sistema debe desarrollarse en **C o C++** y estructurarse en los siguientes módulos fundamentales:

Módulo 1: Gestión de Memoria Virtual y Paginación bajo Demanda (`mmap` y `SIGSEGV`)

- **Reserva de Memoria:** Al iniciar, cada nodo o programa debe reservar un bloque grande de memoria mediante `mmap` (ej. $1\ MB$, o lo que esté configurado o sea indicado por el usuario). Cada nodo que participa en el sistema aportará esta cantidad de "memoria" al sistema. Entre más nodos se agreguen, más memoria habrá disponible y más nodos posibles para trabajo.
- **Control de Permisos:** Inicialmente, las páginas que no pertenecen al nodo local o que están marcadas como inválidas deben configurarse con protección `PROT_NONE` mediante la llamada al sistema `mprotect`.
- **Manejador de Fallos (`SIGSEGV`):**
    - Se debe registrar un manejador de señales usando `sigaction` con la bandera `SA_SIGINFO`.
    - Cuando un hilo intente leer o escribir en una dirección virtual con protección `PROT_NONE` o `PROT_READ` (en intento de escritura), el kernel generará una señal `SIGSEGV`.
    - El manejador debe capturar la dirección exacta que causó el fallo (`siginfo_t.si_addr`), determinar a qué número de página corresponde, pausar la ejecución del hilo afectado y disparar el protocolo de red para obtener la página y los permisos correctos.
- **Parámetros y Configuración:** Se deberá contar con un archivo general de configuración el cual será compartido y leído por todos los nodos en el sistema para que todos usen los mismos valores y parámetros. En este archivo se deberán definir todos los parámetros del sistema: tamaño de página (default 4 kb), cantidad de memoria inicial al reservar por un nodo al unirse al sistema, las direcciones IP o nombres de las computadoras disponibles en la red, puerto de comunicación principal, otros puertos requeridos (control, heartbeat, etc.). Acá deberá agregar y documentar todos los valores que el sistema usará como referencia para su configuración y ajuste en cada ejecución.
- **API:** Para la implementación de las aplicaciones para prueba y validación del sistema DSM implementado, deberá definir un API de uso para el DSM. Las aplicaciones seleccionadas para pruebas deberán implementarse haciendo uso de este API. Documente el listado completo de sus funciones, tipos, parámetros, uso, entradas, salidas y errores.

Módulo 2: Capa de Red y Protocolo de Mensajería (Sockets TCP)

- Cada nodo debe abrir un puerto de escucha TCP para recibir solicitudes de otros nodos.
- Se debe implementar un hilo independiente (**Network Listener Thread**) dedicado exclusivamente a escuchar y atender las peticiones de red concurrentes mediante **Sockets**.
- Implemente un Protocolo de comunicación entre nodos. Documente el detalle de todos los mensajes internos del sistema, sus usos, significados, parámetros, entradas, salidas y posibles errores asociados.

- **Tipos de Mensajes mínimos requeridos:**
    - `PAGE_REQUEST`: Solicitud de una página a su propietario o nodo poseedor.
    - `PAGE_DATA`: Envío del contenido de una página junto con el estado del permiso.
    - `INVALIDATE`: Orden enviada a otros nodos para invalidar su copia local de una página cuando un nodo va a escribir en ella.

Módulo 3: Concurrencia, Hilos y Sincronización Local

- **Hilos de Usuario (Worker Threads):** El programa de prueba principal ejecutará múltiples `pthreads` que realizarán operaciones de lectura y escritura sobre el espacio de memoria compartido. Es decir, los programas para pruebas tales como: sumas de matrices distribuidas, incrementos concurrentes de arreglos, almacenar un archivo grande en memoria de forma distribuida, como un vídeo grande.
- **Protección de Estructuras de Control:** Las tablas de estado de las páginas (que indican el estado MSI de cada página y la lista de copias) deben protegerse estrictamente usando **Semáforos POSIX** (`sem_init`, `sem_wait`, `sem_post`) y mutexes para evitar condiciones de carrera entre los hilos de usuario que fallan por memoria y el hilo de red que recibe mensajes asíncronos.

4. Escenarios para Pruebas y Demostración

El sistema debe probarse ejecutando al menos **dos tipos de aplicaciones diferentes e independientes.** Los programas deberán correrse en terminales distintas o computadoras en la red.

Investigue sobre posibles problemas que son útiles de resolver usando grandes cantidades de memoria; localice 2 implementaciones para este tipo de aplicaciones. Impleméntelas adaptándolas para que puedan ser usadas por el sistema DSM de este proyecto. Es decir, el código de estas aplicaciones debe ser adaptado para que use el API de sus sistema DSM y no las peticiones regulares al sistema operativo.

Demuestre con estas 2 aplicaciones la funcionalidad del sistema DSM implementado. Documente varios escenarios donde utilicen amplias cantidades de memoria. Documente el uso de esta memoria en los múltiples nodos del sistema. Muestre datos de prueba, estadísticas, rendimiento, problemas presentados, etc.

Realice pruebas usando múltiples computadoras físicas. El proyecto será probado con un demo en vivo en los laboratorios de la Escuela de Ing. en Computación en Cartago. Lab H o similar. Se reservarán de 1 a 2 horas para las pruebas de cada proyecto por pareja. Ambas personas deben estar presentes durante el demo en vivo. Ambas personas deberán poder explicar todas y cada una de las partes del sistema implementado. La nota será individual y estará basada en el cumplimiento de esta especificación y la defensa que cada persona brinde sobre lo realizado.

5. Entregables y Criterios de Evaluación

1. **Código Fuente Documentado:** Estructurado en módulos limpios (`dsm.c`, `network.c`, `pager.c`, `main.c`, etc.). Realice un `Makefile` para el proyecto. Documente cómo se debe configurar, instalar y ejecutar su sistema. Brinde un enlace a su repositorio en Git. En el sistema Git deben poder validarse las entradas de commits de cada una de las personas autoras durante las semanas de trabajo de este proyecto.
2. **Prueba de Estrés / Aplicación de Validación:** Las aplicaciones cliente que deben ejecutarse en el sistema usando el espacio DSM, demostrando que los datos se sincronizan correctamente entre las distintas terminales/redes. Estas son ejecuciones reales de aplicaciones reales. Se deben usar datos reales y generar las salidas esperadas según las aplicaciones de prueba seleccionadas. Esta es la forma de demostrar el funcionamiento correcto del sistema DSM implementado. Las pruebas se realizarán en las computadoras de los laboratorios de la Escuela de Ing. en Computación y no en computadoras personales, SIN EXEPCIÓN.
3. **Documentación Técnica (Informe):** Explicación total de todo el sistema implementado, módulos, partes, diagramas, diagrama de estados (MSI), explicar el manejo de la concurrencia entre señales y sockets, y cómo resolvieron los posibles interbloqueos (*deadlocks*) en la red, uso de semáforos, recursos, uso de señales del sistema, estructuras de datos, archivos, detalle del API implementado, detalle del protocolo de mensajería entre nodos, detalle de las aplicaciones de prueba seleccionadas, pruebas de uso y ejecución, manual de usuario, etc.
4. Para procurar el buen desarrollo de este proyecto tendremos **varias entregas semanales** hasta terminar el Proyecto con el Demo en Vivo en los laboratorios de la Escuela de Ing. en Computación.
    a. **V.06.Nov. Entregable 1:** Documento de Diseño del sistema DSM. Arquitectura general, borrador de API inicial, protocolo de mensajes, datos de configuración. Incluya detalles de implementación tales como estructuras y tipos de datos, tamaños de los datos, etc.
    b. **V.13.Nov. Entregable 2:** Prototipo 1 y Arquitectura actualizada. Primera implementación de la arquitectura inicial propuesta y documentación de los cambios o ajustes realizados a la primera versión.
    c. **V.20.Nov. Entregable 3:** Prototipo 2 y Arquitectura actualizada. Implementación mejorada del prototipo inicial propuesto y documentación de los cambios o ajustes realizados para esta segunda versión.
    d. **V.27.Nov. Entrega Final:** Demo en Vivo en el Laboratorio mediante citas de revisión.